

Git, GitHub & Actions, as the City's Diary

TRACKING EVERY CHANGE, SHARING IT, AND REACTING TO IT AUTOMATICALLY

THE BIG PICTURE

Git is the architect's own diary — recording every change ever made to the blueprint. **GitHub** is the shared library where everyone's diaries live, so the whole team works from the same copy. **GitHub Actions** are automatic workers stationed at that library, springing into action the moment something specific happens — like a new entry being dropped off.

GIT —the architect's diary of every change

Records the entire history of the blueprint — every version ever saved, never lost, always possible to go back to.

Example: 5 saved changes over a month means 5 full versions Git remembers — not just the latest one.

STAGING —the draft pile before an entry is written

Lets you choose exactly which changes go into your next diary entry, instead of saving everything blindly.

Example: Two things changed — only the finished one goes in the draft pile and gets committed; the unfinished one waits.

.GITIGNORE —a list of things never written into the diary

Some files should never be tracked at all — passwords, temporary files, personal settings. A .gitignore file tells Git to simply ignore them, so they're never accidentally staged or committed.

Example: A file listing database passwords is added to .gitignore, so it can never accidentally end up shared on GitHub for everyone to see.

COMMIT —one diary entry

A snapshot of exactly what the blueprint looked like at one meaningful moment, with a note explaining what changed.

Example: "Added apartment block C" is one commit. "Fixed road between A and B" is a separate one.

BRANCH —a separate draft copy of the blueprint

Lets you experiment freely without touching the main, trusted blueprint everyone else relies on.

Example: A "new-district-design" branch tries a totally different layout — if it fails, the main blueprint was never at risk.

MERGE —bringing a branch back into the main diary

Combines a branch's changes into the main blueprint. If two people changed the same part differently, that's a merge conflict — a human has to decide.

Example: The successful district redesign branch gets merged in, and is now part of the official blueprint.

REMOTE —a copy of the diary stored elsewhere

Keeps the diary from being stuck on just one person's desk. You push changes up to it, and pull others' changes down.

Example: The architect pushes finished changes to the remote copy so the whole team sees the latest version too.

PULL REQUEST —formally asking for your draft to be reviewed and approved

In real teams, a branch usually isn't merged straight away — a Pull Request is a formal request on GitHub asking someone else to review the changes first, before they're allowed into the main blueprint.

Example: The architect pushes the "new-district-design" branch, then opens a Pull Request asking a senior architect to review it. Only once approved does it get merged into the main blueprint.

CLONE —getting your own personal copy of the diary

Downloads the entire diary — every commit, every branch — onto your own machine, ready to work from immediately.

Example: A new architect joins and clones the GitHub repository instead of being emailed files.

GITHUB —the shared library where every diary lives

A website that hosts remotes — plus extra tools on top, like reviewing changes before merging, and reacting automatically when something changes.

Example: The whole team pushes and pulls from one shared GitHub repository instead of emailing files around.

GITHUB ACTIONS —automatic workers stationed at the library

Watch the shared library and spring into action the moment something specific happens — a fully automated worker, built directly into GitHub.

Example: A worker automatically runs tests the moment new code is pushed, with no one triggering it by hand.

TRIGGER —the specific event that wakes a worker up

Workers don't run randomly — they wait for a specific event: a push to a branch, a pull request, or a schedule.

Example: "On push to develop" wakes the worker, which then runs the pipeline — test, build, deploy.

Staging (draft pile) → Commit (diary entry) → Branch (separate draft)

→ Remote (copy elsewhere) → Pull Request (ask for review) → Merge (combine back in)
→ Clone (your own copy) → GitHub (shared library)
→ Trigger (the event) → GitHub Actions (the automatic worker that responds)

Turn a folder into a diary GIT — THE DIARY

```
git init
```

Starts a brand-new **diary (Git repository)** in the current folder.

Check the current draft state GIT — THE DIARY

```
git status
```

Shows what's currently staged, unstaged, or untracked — a quick check of exactly where things stand before you commit anything.

Put changes in the draft pile STAGING — DRAFT PILE

```
git add filename.txt
```

```
git add .
```

Marks specific changes as ready for the next **entry (commit)**. The second version stages everything changed at once.

Write the diary entry COMMIT — DIARY ENTRY

```
git commit -m "Added apartment block C"
```

`-m "..."` — the note explaining what this entry changed

Permanently saves the staged changes as one named **diary entry (commit)**.

View the diary's history COMMIT — DIARY ENTRY

```
git log
```

```
git log --oneline
```

Shows every past **commit**, in order — who made it, when, and its note. The "--oneline" version condenses each entry to a single line.

Start a separate draft BRANCH — SEPARATE DRAFT

```
git branch new-district-design
```

```
git checkout new-district-design
```

Creates a new **branch**, then switches into it — changes made here won't touch the main blueprint.

Bring the draft back in MERGE — COMBINE BACK IN

```
git checkout main
```

```
git merge new-district-design
```

Switches back to the main diary, then folds the **branch's** changes into it.

Connect a new local diary to GitHub REMOTE — COPY ELSEWHERE

```
git remote add origin https://github.com/user/repo.git
```

`origin` — the name Git gives this remote by default

Links a diary that started locally (via **git init**) to a GitHub repository, so it has somewhere to push to for the first time.

Share your changes REMOTE — COPY ELSEWHERE

```
git push origin main
```

```
git pull origin main
```

Push sends your local commits up to the **remote**. Pull brings everyone else's commits back down to you.

Ask for your draft to be reviewed PULL REQUEST — REQUEST REVIEW

```
gh pr create --base main --head new-district-design
```

`--base main` — the main diary you want your changes merged into
`--head new-district-design` — your branch, containing the changes to review

Opens a **Pull Request** on GitHub. This is usually done through GitHub's website directly, but the "gh" command-line tool can do it too. Someone reviews it, and only once approved does it actually get merged.

Get your own copy

CLONE — YOUR OWN COPY

```
git clone https://github.com/user/repo.git
```

Downloads the entire **diary** — every commit, every branch — from GitHub onto your own machine.

A GitHub Actions workflow file

ACTIONS — AUTOMATIC WORKER

```
name: Run Tests

on:
  push:
    branches: [develop]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run tests
        run: npm test
```

on: push: [develop] — the trigger: wake up on a push to develop
jobs: test: — the actual job the worker runs once woken
runs-on: ubuntu-latest — which machine the worker uses to do the job
steps: ... — the individual actions taken, in order

Saved as a file inside the repository (.github/workflows/). The moment someone pushes to develop, this worker wakes up and runs the test job automatically — no one has to trigger it by hand.

JOBS & STEPS —the crew, and their task list

A workflow can have multiple jobs (e.g. "test," "build," "deploy"), each made of steps run in order. Jobs run in parallel by default — unless one is told to wait for another to finish first.

Example: A "deploy" job waits for the "test" job to succeed first — the same pass-before-proceed gate used in a Jenkins pipeline.

RUNNER —the actual machine doing the work

Every job needs somewhere to execute. GitHub provides free runners, but companies often use their own for extra control, security, or to reach internal infrastructure the public ones can't.

Example: A job that needs to deploy inside a private company network runs on a self-hosted runner instead of a public GitHub one.

SECRETS —passwords and keys, kept out of sight

Real pipelines need credentials — cloud keys, database passwords — but never hardcoded into the workflow file for anyone to read. Secrets are stored securely in GitHub's settings and injected only at runtime.

Example: An AWS key is stored as a secret and referenced as `${{ secrets.AWS_KEY }}` — the actual value never appears anywhere in the file itself.

ENVIRONMENTS & APPROVALS —a human checkpoint before production

A workflow can require someone to manually approve a deployment before it's allowed to proceed — an extra safety gate on top of the automatic trigger, usually reserved for production.

Example: Tests pass automatically, but deploying to production still waits for a senior engineer to click "approve."

ARTIFACTS —passing results between jobs

Each job starts on a fresh, empty runner — so if one job builds something and another needs it, the result has to be explicitly passed along as an artifact.

Example: A "build" job produces a packaged app; a later "deploy" job downloads that same artifact to actually deploy it.

REUSABLE WORKFLOWS —a packaged pipeline, shared across projects

The same idea as a Terraform module or an Ansible role — a workflow can be packaged once and reused by multiple projects, instead of every project duplicating the same pipeline definition.

Example: A standard "test and deploy" workflow is defined once, and every team's project simply calls it instead of rewriting it.

A fuller, realistic pipeline **JOBS, SECRETS & APPROVALS TOGETHER**

```
name: Deploy Infrastructure

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm test

  plan:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: terraform plan
    env:
      AWS_ACCESS_KEY_ID: ${ secrets.AWS_KEY }

  apply:
    needs: plan
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: terraform apply -auto-approve
    env:
      AWS_ACCESS_KEY_ID: ${ secrets.AWS_KEY }
```

needs: test — this job waits until "test" succeeds before running
secrets.AWS_KEY — a securely stored credential, never visible in the file
environment: production— requires manual approval before this job is allowed to run

Three jobs, each depending on the last: **test** runs first, **plan** only runs if tests pass, and **apply** only runs after a human approves the production environment. This is genuinely close to what a real infrastructure pipeline looks like.

HOW THIS CONNECTS TO EVERYTHING ELSE

What relies on what: Git needs nothing else to work — it's just file history on your machine. GitHub relies on Git, since it's a website built to host Git repositories. GitHub Actions relies on GitHub, since it watches for events happening inside a repository there. Terraform doesn't strictly need any of these — it can run entirely by hand — but in an automated pipeline, its blueprint is stored in Git and triggered by Actions. Kubernetes relies on Terraform, since the cluster has to be built before Kubernetes has anything to manage inside it.

The actual order of events in a real pipeline: a file changes → **Git** records that change → it's pushed to **GitHub** → **GitHub Actions** notices the push and wakes up → it runs **Terraform**, which builds or updates the infrastructure, including the cluster → **Kubernetes** then manages what runs inside that cluster. Git itself never touches the cluster directly — it's several steps removed.

QUICK SELF-TEST

1. What's the difference between staging and committing?
2. Why do teams use branches instead of editing the main version directly?
3. What's a merge conflict, and why does it happen?
4. What's the difference between Git and GitHub?
5. What actually triggers a GitHub Actions worker to run?
6. What actually runs once a GitHub Actions worker is triggered?

IF THE MANAGER ASKS...

"Why use branches instead of just editing directly?"

So multiple people can work in parallel without stepping on each other, and the main version stays stable and trustworthy at all times.

"What's the difference between GitHub Actions and Jenkins?"

Same underlying idea — automatically trigger a pipeline on a Git event — but GitHub Actions is built directly into GitHub, while Jenkins is a separate standalone tool.

"What's the difference between Git and GitHub?"

Git is the tool that tracks version history, on your own machine. GitHub is the website that hosts and shares those repositories with a team, plus extra collaboration tools on top.

Quick reference

Git	— the diary tracking every change
Staging	— the draft pile before committing
.gitignore	— files deliberately never tracked
Commit	— one saved diary entry
Branch	— a separate draft copy
Pull Request	— asking for a branch to be reviewed
Merge	— combining a branch back in
Remote	— a copy stored elsewhere
Clone	— your own personal copy
GitHub	— the shared library hosting repositories
GitHub Actions	— automatic workers reacting to events
Trigger	— the specific event that wakes a worker up